Copyright

Warning
This material has been reproduced and communicated to you by or on behalf of Federation University Australia in accordance with Section 113P of the Copyright Act 1968 (the Act).

The material in this communication may be subject to copyright under the Act. Any further reproduction or communication of this material may be the subject of copyright protection under the Act.

Do not remove this notice.

ITECH3102 –Data Visualization
Week 2

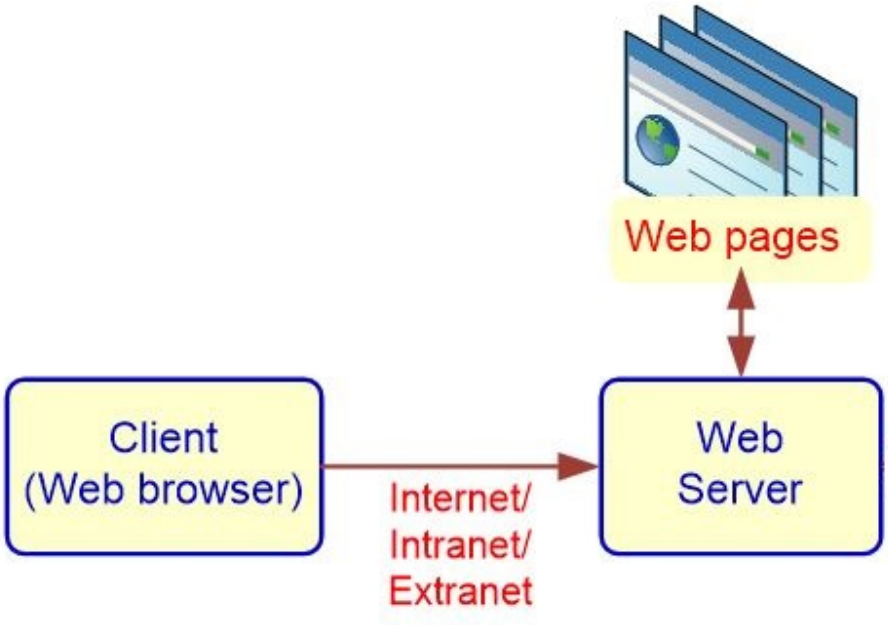
Lab Exercises –Fundamentals of Web Technology: HTML, CSS

ID: _____Name: _____

In this course, we also use D3 (Data-Driven Documents or D3.js called JavaScript library) for visualizing data. D3 makes use of Scalable Vector Graphics (SVG), HTML, and Cascading Style Sheets (CSS) standards to produce dynamic, interactive data visualizations in web browsers.
In this week's projects, we mainly discuss some fundamentals of web technology including HTML, CSS. In the next week, we explore JavaScript and D3 for visualizing data.

Introduction of Web

We think of the web as a bunch of interlinked pages, but it's really a collection of conversations between web servers and web clients (browsers).



Clients contact servers with requests, and servers respond with data.

- Web servers are internet-connected computers running server software, so called because they serve web documents as requested. Servers are typically always on and always connected, but web developers often also run local servers, meaning they run on the same computer that you're working on. Local means here. In particularly, we use an HTTP server that is a computer (software) program to play the role of a server in a client–server model.
- In contrast, a web browser (such as Chrome, Firefox, etc.) is application software for accessing the World Wide Web or a local website. Clients contact servers with requests. For example, when a user requests a web page from a particular website, the web browser retrieves the necessary content from a web server and then displays the page on the user's device.

Every web page, in theory, can be identified by its URL (Uniform Resource Locator) or URI (Uniform Resource Identifier). A complete URL, then, might look like this: https://federation.edu.au.

Conversations between web servers and web clients (browsers)

1. User runs the web browser, then types a URL into the address bar.
2. The browser then attempts to connect to the server
3. The server associated with the request acknowledges the connection and is taking requests.
4. The browser sends a request for the page that lives at server

5. The server sends back the HTML content for that page.
6. As the client browser receives the HTML, it discovers references to other files needed to assemble and display the entire page, including CSS stylesheets and image files. So it contacts the same server again, once per file, requesting the additional information.
7. The server responds, dispatching each file as needed.
8. Finally, all the web documents have been transferred over. Now the client (browser) renders the content. It first parses through the HTML to understand the structure of the content. Then it reviews the CSS selectors, applying any properties to matched elements. Finally, it plugs in any image files and executes any JavaScript code.

Project 1: HTML

1 Hypertext Markup Language (HTML)

Hypertext Markup Language (HTML) is not a procedural programming language like C. Hypertext Markup Language (HTML) is a markup language for identifying the elements of a Web page so that a browser can render that page on computer screen. HTML is used to format text. This marking up of information is different from the intent of traditional programming languages like C, which is to perform actions in a designated order.

(1) HTML tags and attributes

In HTML, text is marked up with elements, delineated by tags that are keywords contained in pairs of angle brackets. For example, the HTML element itself, which indicates that we are writing a Web page to be rendered by a browser, begins with a start tag of <HTML> and terminates with an end tag of </HTML>.

<HTML>

</HTML>

(2) Software for Editing HTML

HTML files are called HTML documents that are just plain-text files. Now create a simple HTML page within your project folder named index.html. Remember, HTML documents are just plain-text files, so you can use the text editor of your choice. Free editors like TextEdit and Notepad are fine, but your life will be easier if you use an editor designed specifically for working with code, like Atom, Brackets, or Sublime Text (among many others). You can use free text editors like Notepad and TextEdit to create HTML documents. In addition, you can also use modern-day browsers that has built-in developer tools to create HTML pages.

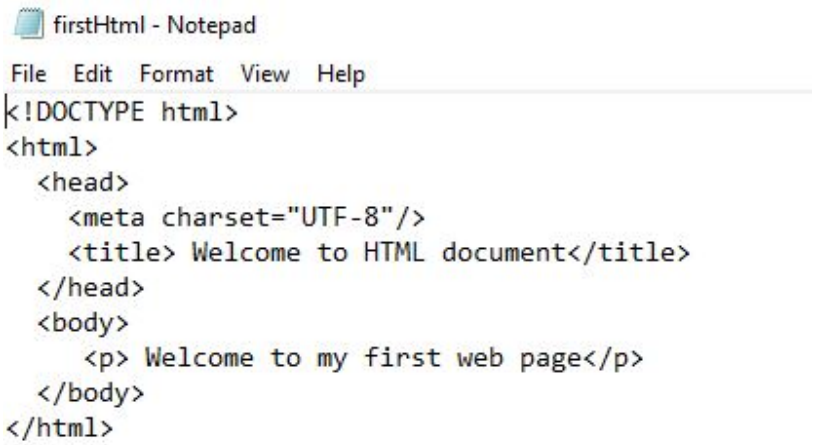
In this course, we use Notepad and browser Chrome's developer tool.

(a) Using Notepad to create HTML page

Notepad is built into Windows operating systems and can be found inside the Windows Accessories panel of your program list, inside the Start menu. All HTML files typically have either the .htm or the .html file name extension. We recommend that you name all of your HTML files with the **.html** extension.

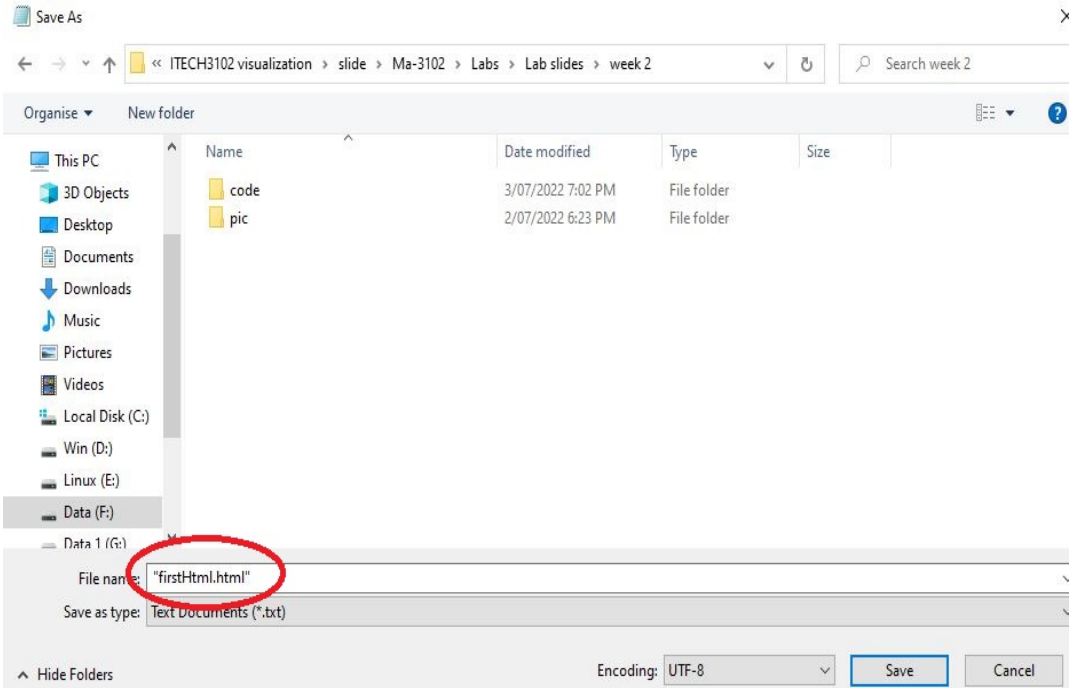
Type the following codes

Open Notepad from Windows Accessories panel in your computer and type the following codes.



Save your HTML file

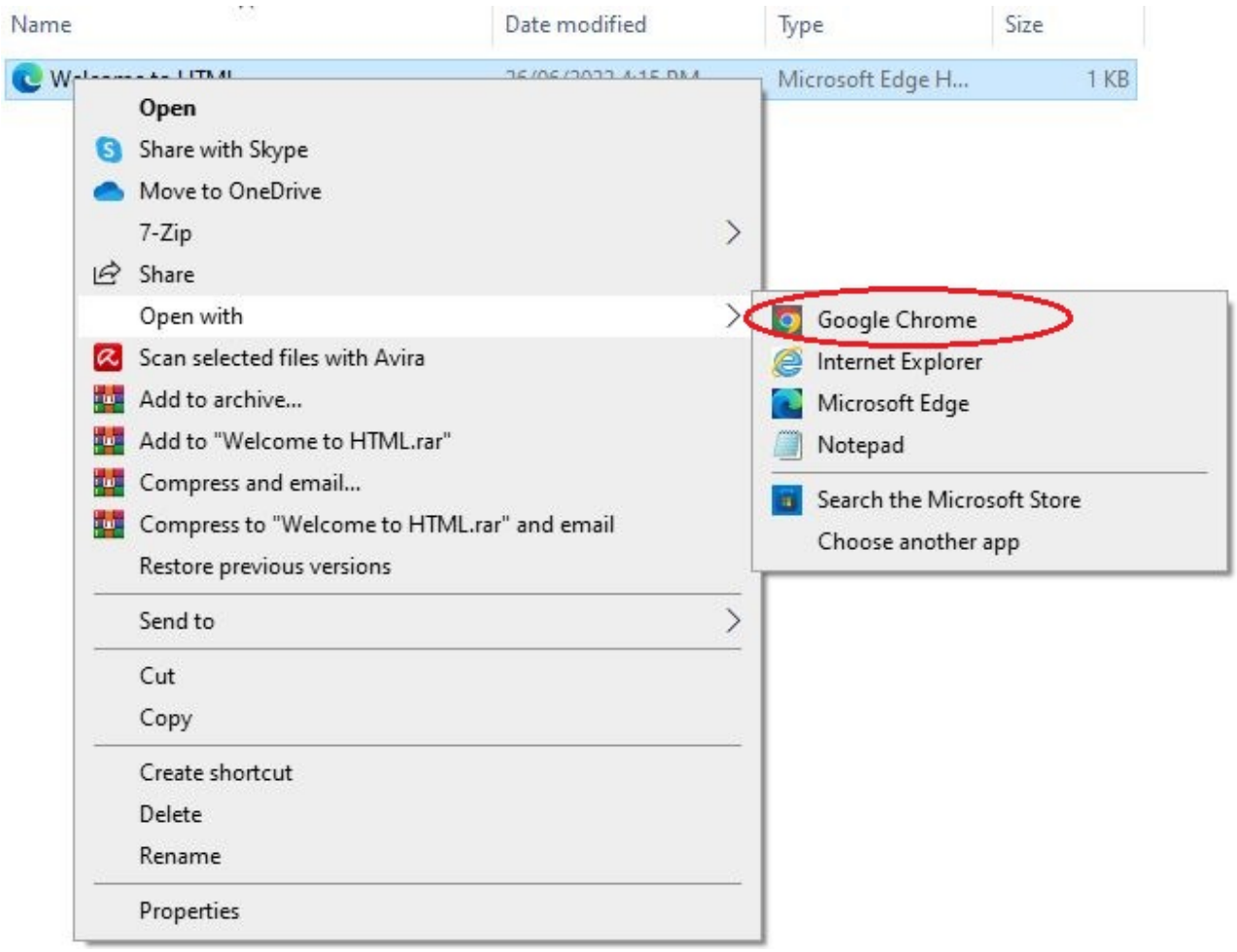
To save above codes as HTML file, you need to Select the File Menu from the Top Navigation Bar in Notepad Text Editor and then click 'Save As', type "firstHtml.html" in the box of File name. Here firstHtml is the name of your file and .html is extension.



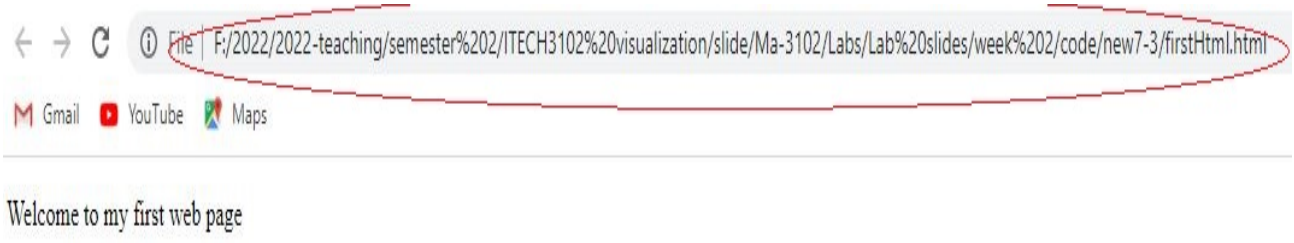
View the firstHtml.html page in your web browser

In this course, we use **Google Chrome** browser to view and edit web pages. However, you can also use other browsers to view web pages, such as using **Microsoft Edge,** etc.

After saving firstHtml.html in your computer, right click the html file and select Google Chrome



You can find that texts are displayed in browser. Please note that the URL path starts with a folder: F:/2022/2022-teaching/…./firstHtml.html. This indicates that the page is not viewed through a web server.



Common Elements:

In the above codes in the html file,
<!DOCTYPE html>
The standard document type declaration. Must be the first thing in the document.

html
Surrounds all HTML content in a document. <html> tells the browser that everything contained between the opening <html> tag and the closing </html> tag is HTML.

head
The document head contains all metadata about the document, such as its title and any references to external stylesheets and scripts. Every HTML file is separated into a header element, which generally contains information about the document, and a body, which contains the page content.

<meta charset=" UTF-8"/>

Meta elements are tags used in HTML to provide structured metadata about a Web page. They are part of a web page's head section. Meta elements can be used to specify page description, keywords and any other metadata not provided through the other head elements and attributes.
UTF-8 is a variable-width character encoding used for electronic communication. UTF-8 encodes each character (code point) in 1 to 4 octets (8-bit bytes).
title
The title of the document. Browsers typically display this at the top of the browser window and use this title when bookmarking a page.

body
Everything not in the head should go in the body. This is the primary visible content of the page. The body of an HTML document is the area where you place all content you would like browsers to display. This includes text, images, links, and forms, etc.
p
Paragraph element
 Welcome to my first web page
All text placed between the . . . tags forms one paragraph.

In addition, two tags
</body>
</html>
close the body and HTML sections of the document respectively.

Other common elements:

Comments in HTML always begin with **<! - - and end with - ->.** The browser ignores any text and /or tags inside a comment.

h1, h2, h3, h4
These let you specify headings of different levels. h1 is a top-level heading, h2 is below that, and so on.

ul, ol, li
Unordered lists are specified with ul, most often used for bulleted lists. Ordered lists (ol) are often numbered. Both ul and ol should include li elements to specify list items.

em
Indicates emphasis. Typically rendered in italics.

strong
Indicates additional emphasis. Typically rendered in boldface.

a
A link. Typically rendered as underlined, blue text, unless otherwise specified.

span
An arbitrary span of text, typically within a larger containing element like p.

div
An arbitrary division within the document. Used for grouping and containing related elements.

Project 2: Using browser's built-in developer tools

In the project 1, we use free text editors like Notepad to create HTML documents. In this project, we use modern-day browsers that have built-in developer tools to view raw source codes of HTML pages and edit HTML files.

Type the following codes

Open Notepad from Windows Accessories panel in your computer and type the following codes.

second - Notepad

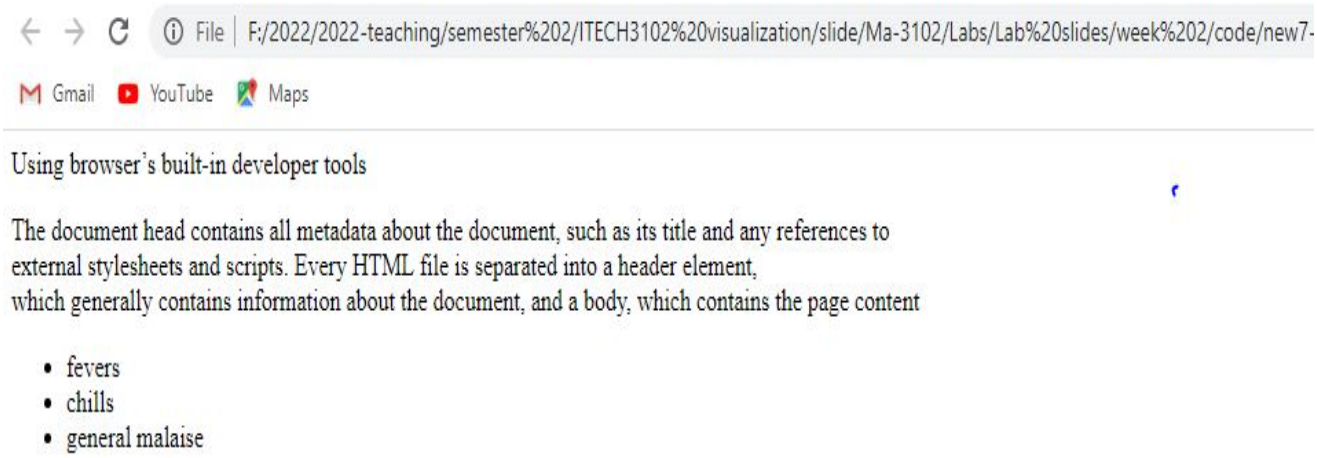
File Edit Format View Help

```
<!DOCTYPE html>
<html>
  <head>
    <meta charset="UTF-8"/>
    <title> Welcome to HTML document</title>
  </head>
  <body>
     <h>Using browser's  built-in developer tools </h1>
     <p> The document head contains all metadata about the document,
         such as its title and any references to <br>external stylesheets and scripts.
         Every HTML file is separated into a header element,
         <br> which generally contains information about the document, and a body,
         which contains the page content</p>
     <ul>
         <li> fevers</li>
         <li> chills</li>
         <li> general malaise</li>
     </ul>


</body>
</html>
```

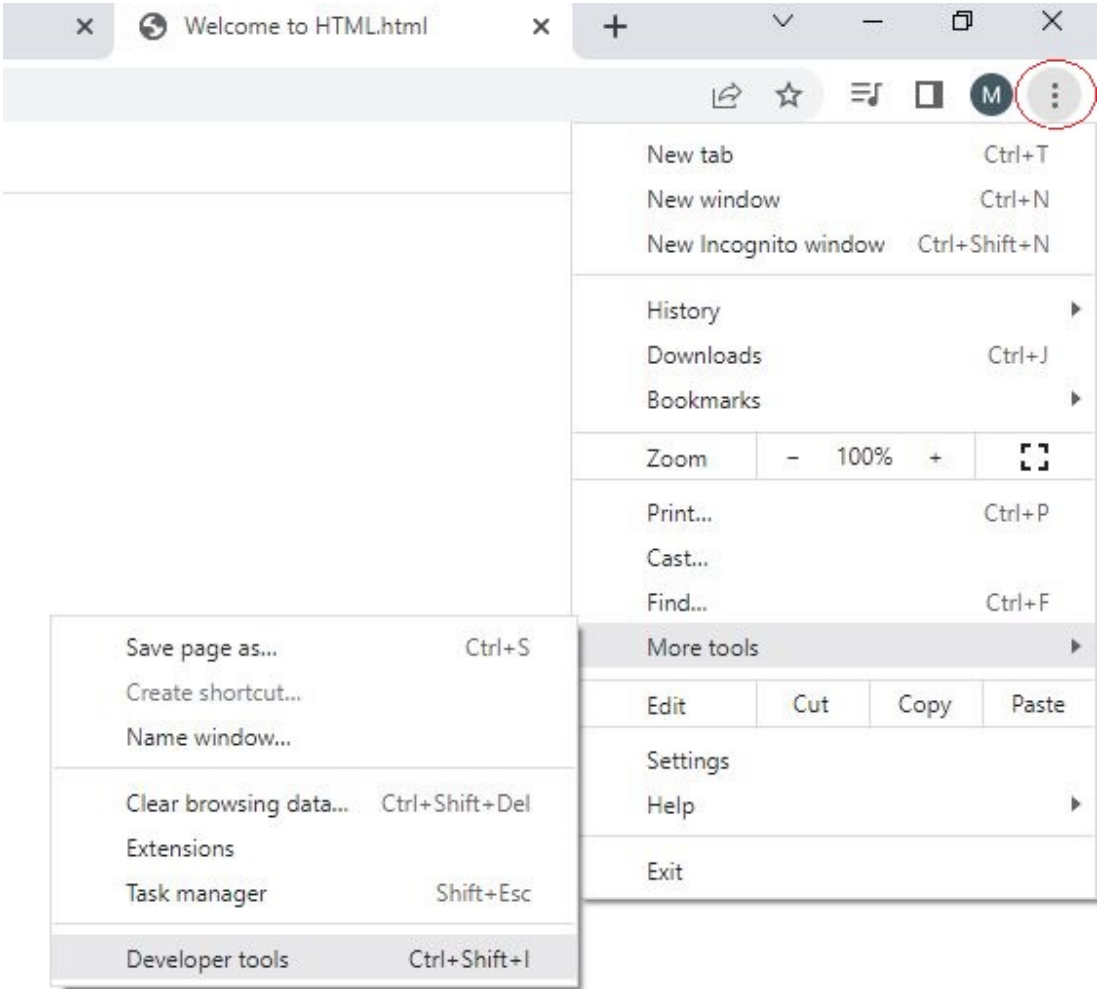
Save your HTML file

After saving this html in your computer, right click the html file and select Google Chrome and you can see the following texts displayed in browser.

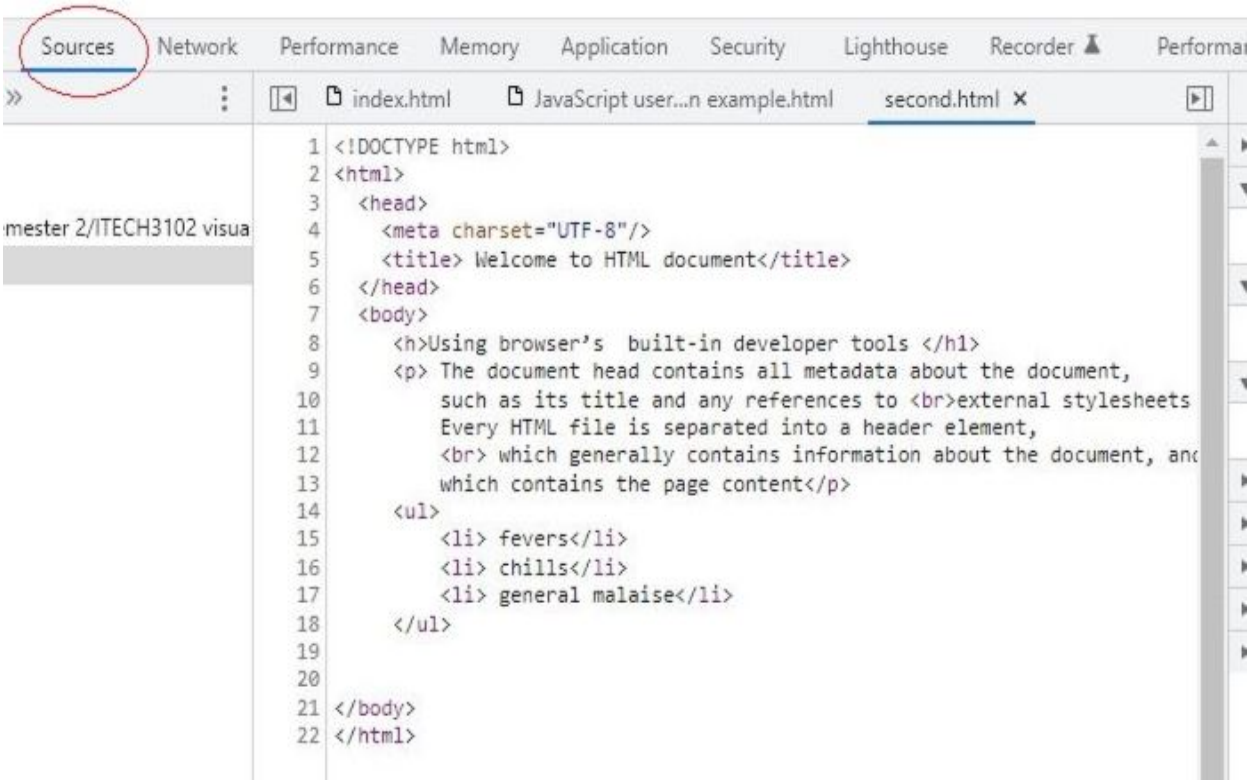


Open Developer tools

Click on the Chrome menu icon on the browser toolbar at the top right. This icon is a box with three vertical dots. Click More tools and click Developer tools to open it.



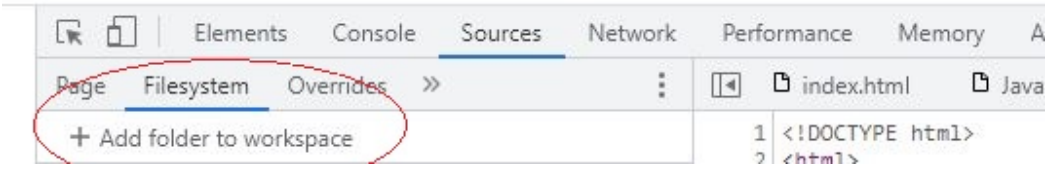
Click Developer tools to open it and click Sources tap. You can view the raw source code of the second.html page. You can also click Elements tab to see the current stage of the DOM.



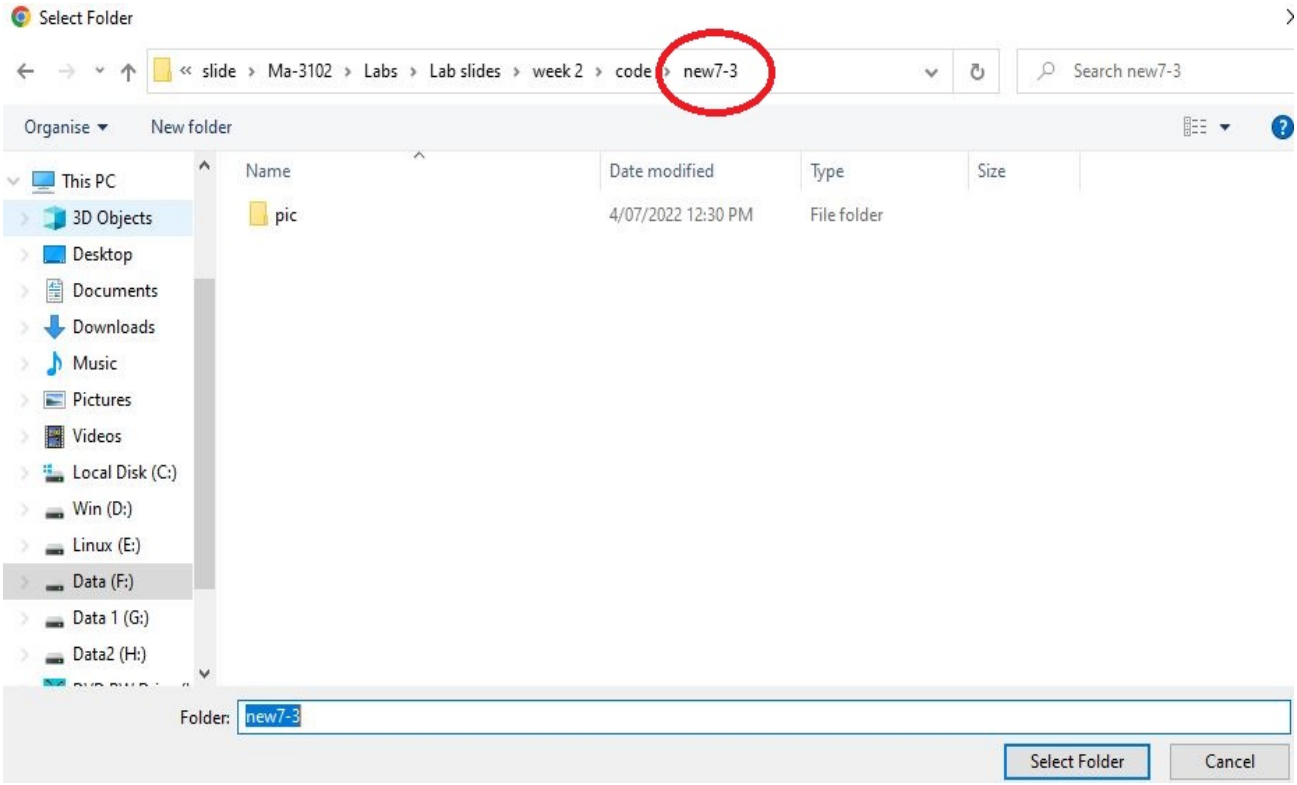
Associate your folder with workspace

Your html file is stored within the folder in your local computer. You need to associate your html file(folder) with the workspace provided by developer tools so that any changes made in your codes will be saved into your html file.

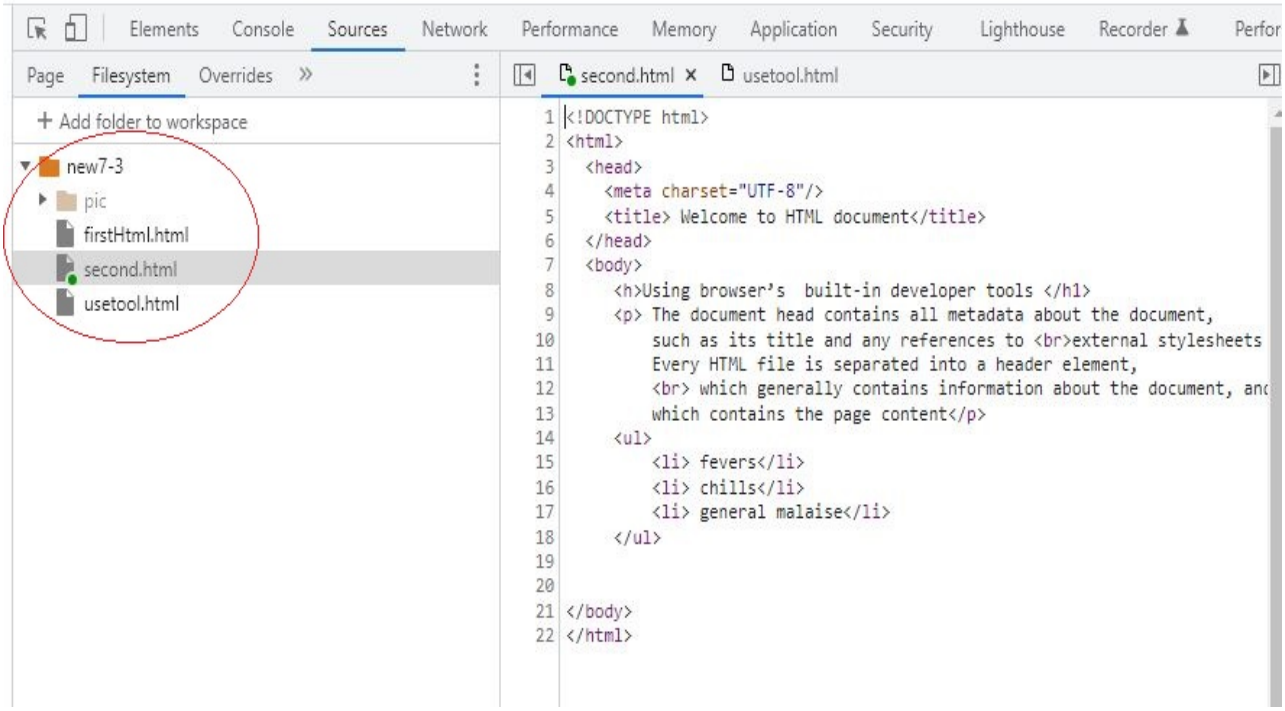
Firstly, click Filesystem tab and then click Add folder to workspace



Select the folder where the html page is stored.

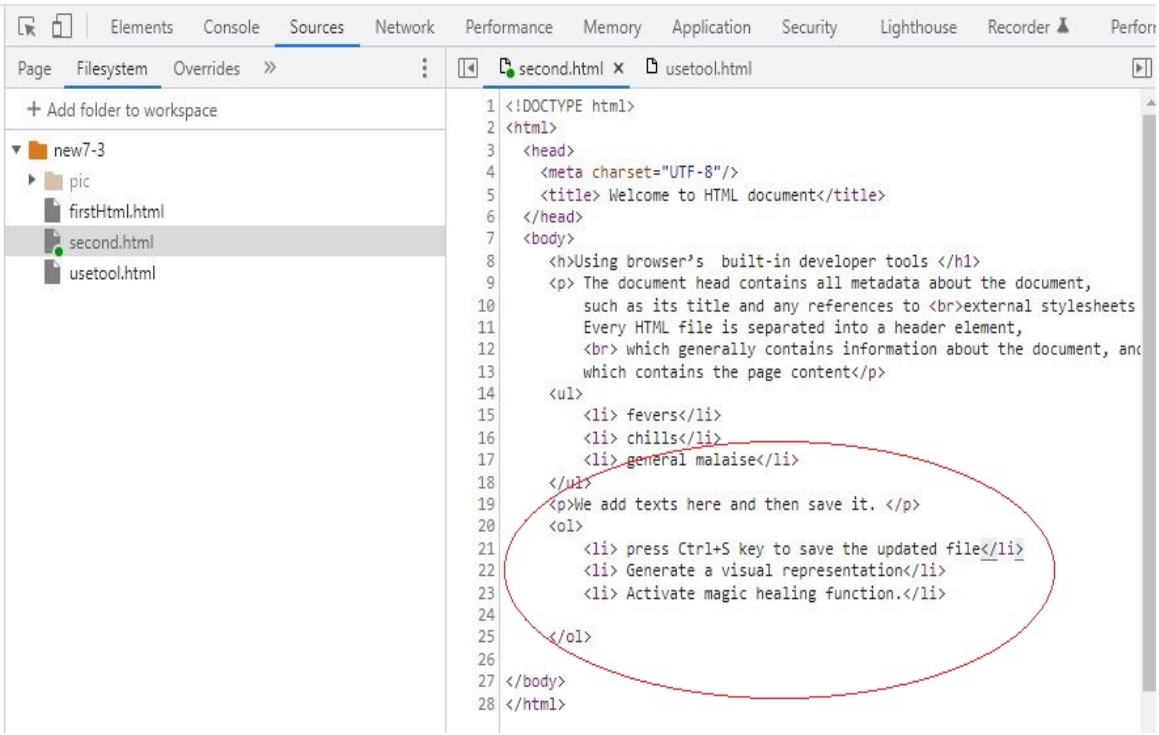


After clicking Select Folder button, the selected folder (e.g, new7-3 in my computer) is added to workspace.

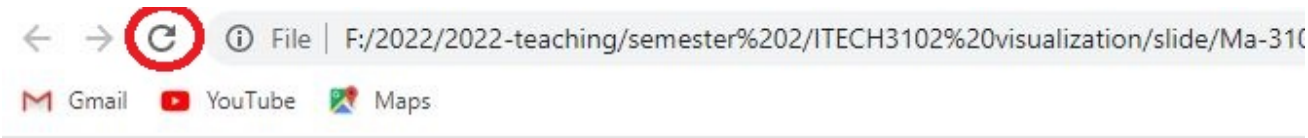


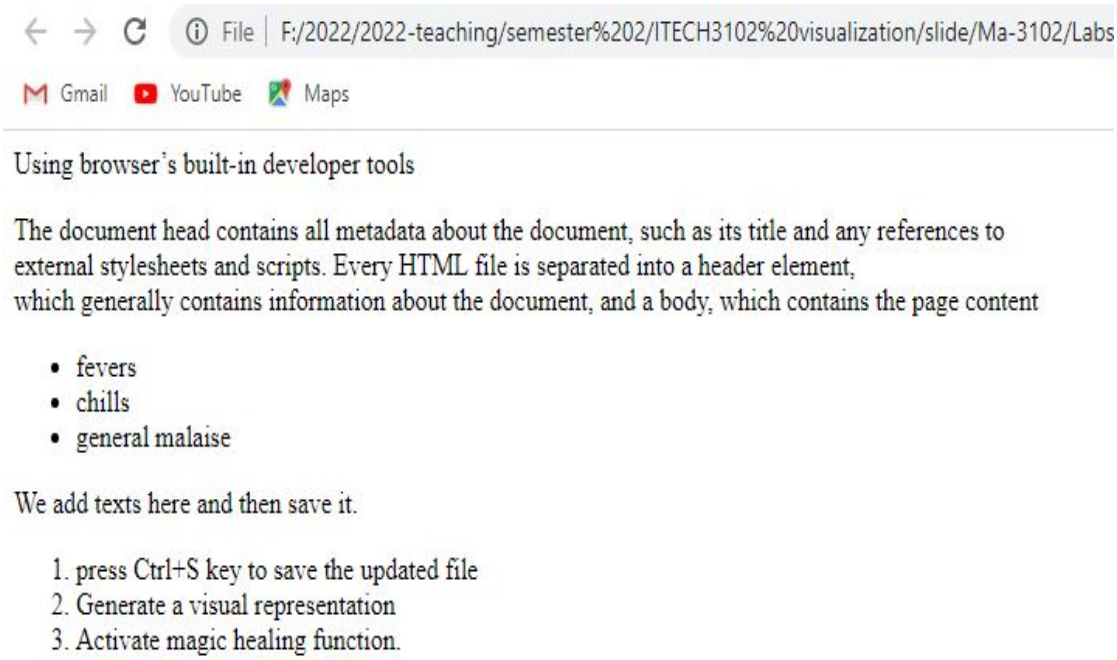
Edit and Save html file.

Add the following codes to the html file and then press **Ctrl + S** keys to save the updated html file.



Then reload the second.html to view updated page.





Other common elements and attributes:

Attributes

We assign all HTML elements attributes by including property/value pairs in the opening tag.

<tagname property="value"></tagname>

For example, the link tag can be given an href attribute, whose value specifies the URL for that link. (href is short for "hypertext reference.")

Federation University Australia

Classes and IDs

Classes and IDs are extremely useful attributes, as they can be referenced later to identify specific pieces of content. Your CSS and JavaScript code will rely heavily on classes and IDs to identify elements. For example:

```
<p>Brilliant paragraph</p>
<p>Insightful paragraph</p>
<p class="awesome">Awe-inspiring paragraph</p>
```

IDs are used in much the same way, but there can be only one ID per element, and each ID value can be used only once on the page. For example:

```
<div id="content">
   <div id="visualization"></div>
   <div id="button"></div>
</div>
```

DOM

The term Document Object Model refers to the hierarchical structure of HTML. Each pair of bracketed tags (or, in some cases, a single tag) is an element, and we refer to elements' relationships to each other in human terms: parent, child, sibling, ancestor, and descendant. For example, in this HTML:

```
<html>
   <body>
      <h1>Breaking News</h1>
      <p></p>
   </body>
</html>
```

body is the parent element to both of its children, h1 and p (which are siblings to each other). All elements on the page are descendants of html.

Project 3 Cascading Style Sheets (CSS)

Cascading Style Sheets (CSS) allow us to specify the style of page elements (spacing, margins, etc.) separately from the structure of our document (section headers, body text, links, etc.). This separation of structure from content allows greater manageability and makes changing the style of document easier.

Cascading Style Sheets are used to style the visual presentation of DOM elements. CSS looks like the following:

```
body {
   background-color: white;
   color: black;
}
```

CSS styles consist of selectors and properties. Selectors are followed by properties, grouped in curly brackets. A property and its value are separated by a colon, and the line is terminated with a semicolon, like the following:

```
selector {
   property: value;
   property: value;
   property: value;
}
```

Selectors

D3 uses CSS-style selectors to identify elements on which to operate, so it's important to understand how to use them.

Selectors identify specific elements to which styles will be applied. There are several different kinds of selectors. We'll use only the simplest ones in this course.

Type selectors

These are the simplest. They match DOM elements with the same name:

```
h1          /* Selects all level 1 headings          */
p          /* Selects all paragraphs               */
strong      /* Selects all strong elements          */
em          /* Selects all em elements              */
div         /* Selects all divs                     */
```

Class selectors

These match elements of any type that have been assigned a specific class. Class names are preceded with a period, as shown here:

```
.caption    /* Selects elements with class "caption"  */
.label      /* Selects elements with class "label"    */
.axis       /* Selects elements with class "axis"     */
```

Because elements can have more than one class, you can target elements with multiple classes by stringing the classes together, as in the following:

```
.bar.highlight  /* Could target highlighted bars      */
.axis.x         /* Could target an x-axis             */
.axis.y         /* Could target a y-axis              */
```

.axis could be used to apply styles to both axes, for example, whereas .axis.x would apply only to the x-axis.

ID selectors
These match the single element with a given ID. (Remember, IDs can be used only once each in the DOM.) IDs are preceded with a hash mark.

```
#header     /* Selects element with ID "header"       */
#nav        /* Selects element with ID "nav"          */
#export     /* Selects element with ID "export"       */
```

Properties and Values

Groups of property/value pairs cumulatively form the styles:

```
margin: 10px;
padding: 25px;
background-color: yellow;
color: pink;
font-family: Helvetica, Arial, sans-serif;
```

Referencing Styles

There are three common ways to apply CSS style rules to your HTML document.

1. **Embed the CSS in your HTML**

If you embed the CSS rules in your HTML document, you can keep everything in one file. In the document head, include all CSS code within a style element.

Example 1:
Type the following codes in text editor like Notepad:

embedCSS - Notepad

File Edit Format View Help

```
<!DOCTYPE html>
<html>
  <head>
    <meta charset="UTF-8"/>
    <title> Using Embed the CSS in HTML</title>
    <style type="text/css">

            p {
                font-size: 24px;
                font-weight: bold;
                background-color: red;
                color: white;
            }

     </style>
  </head>
  <body>

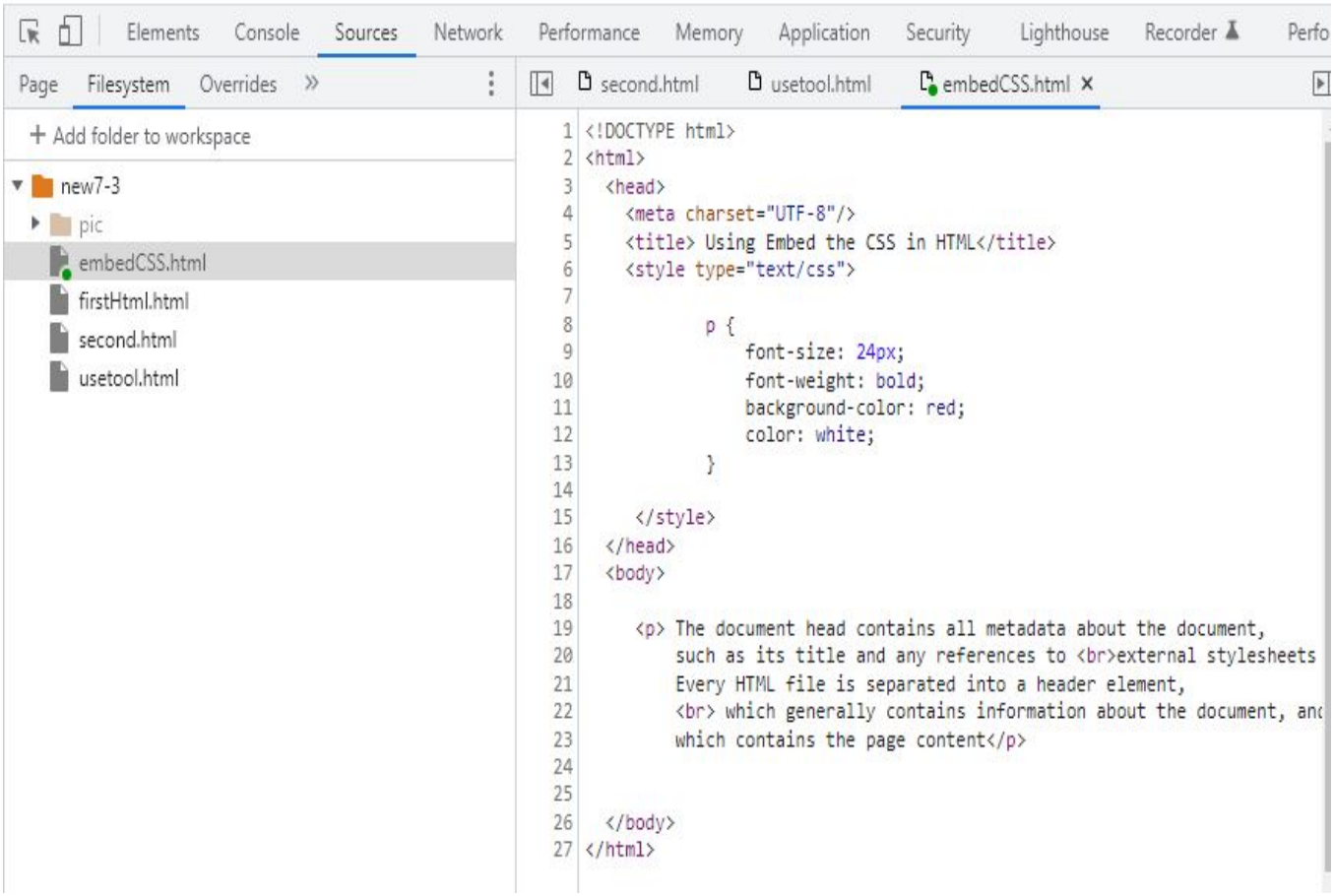
     <p> The document head contains all metadata about the document,
         such as its title and any references to <br>external stylesheets and scripts.
         Every HTML file is separated into a header element,
         <br> which generally contains information about the document, and a body,
         which contains the page content</p>


  </body>
</html>
```

After saving the html file and the opening the html file by using Chrome, you'll get:



You can also view source codes by using Developer tools in Chrome:



Example 2:

Type the following codes in text editor like Notepad:

embedCSS-1 - Notepad

File Edit Format View Help

```
<!DOCTYPE html >
<html>
  <head>
    <meta charset="UTF-8"/>
    <title>  Cascading Style Sheets example  </title>
    <style type  ="text/css">
         EM { background-color:#8000FF;
              color: white }
         H1 {font-family: Arial, sans-serif }

         P {font-size: 18pt }

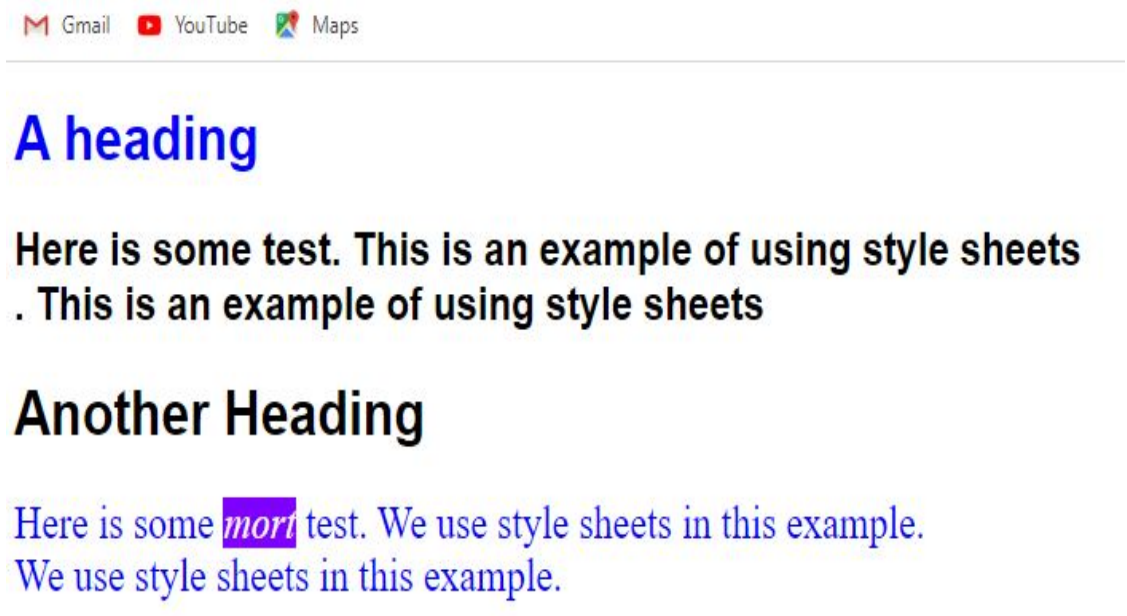
        .blue {color: blue}

     </style>
  </head>
  <body>
    <H1 class = "blue"> A heading <H1>
    <P> Here is some test. This is an example of using style sheets<br>.
     This is an example of using style sheets</P>

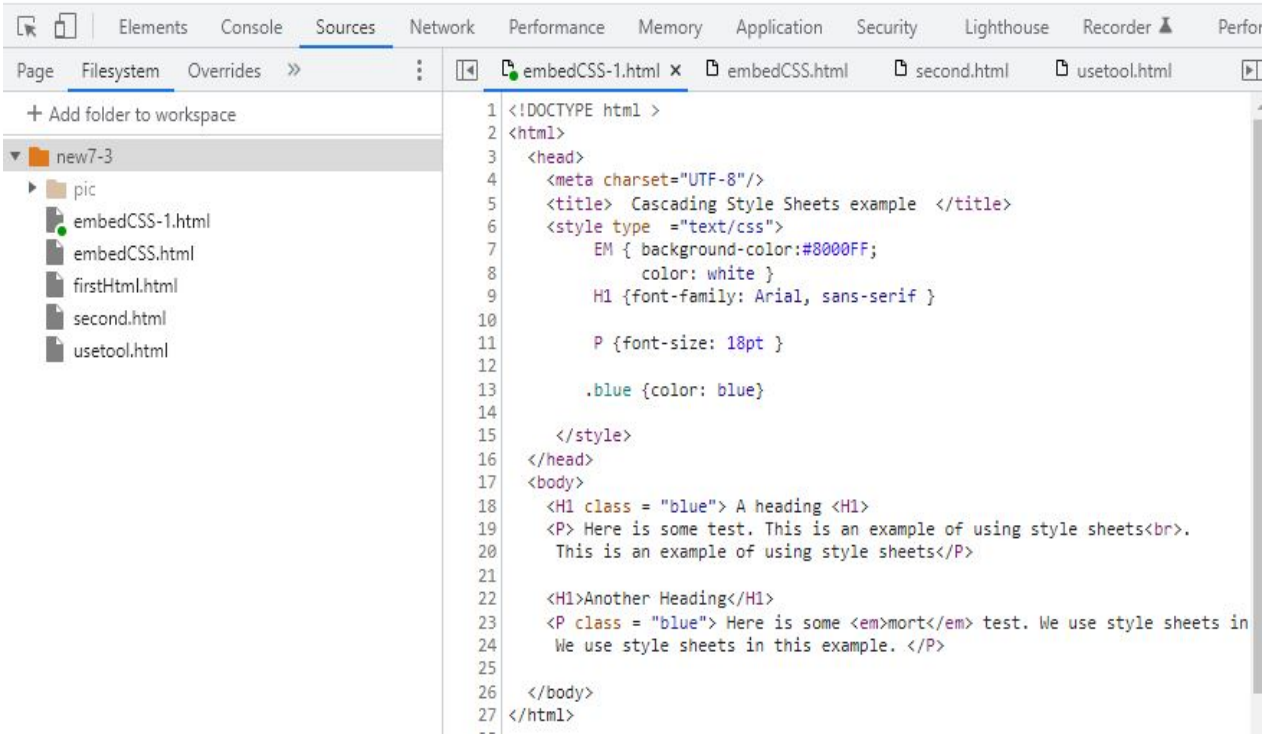
    <H1>Another Heading</H1>
    <P class = "blue"> Here is some <em>mort</em> test. We use style sheets in this example.<br>
     We use style sheets in this example. </P>

  </body>
</html>
```

After saving the html file and the opening the html file by using Chrome, you'll get:



You can also view source codes by using Developer tools in Chrome:



In the above codes, line 6 begins the style sheet. Styles that are placed here apply to the whole document. The type attribute specifies the MIME types of style sheets. Regular text style sheets use the MIME type of text/css.

The body of the style sheet on line 7 through 13 declares the **CSS rules** for this style sheet. We declare rules for the EM, H1, and P elements. All EM, H1, and P elements in this document will be modified in the specified manner. Each rule body begins and ends with a curly brace ({and}). We also declare a style class named blue on line 13. All class declarations are preceded with a period and are applied to elements only of that specific class.

The CSS rules use format: the property is followed by a colon (**:**) and the value of that property.

On line 18, the class attribute applies a tyle class, in this case blue (this was declared as .blue in the style sheet).

On lines 23 and 24, the P element and the .blue class style are both applied to the whole text.

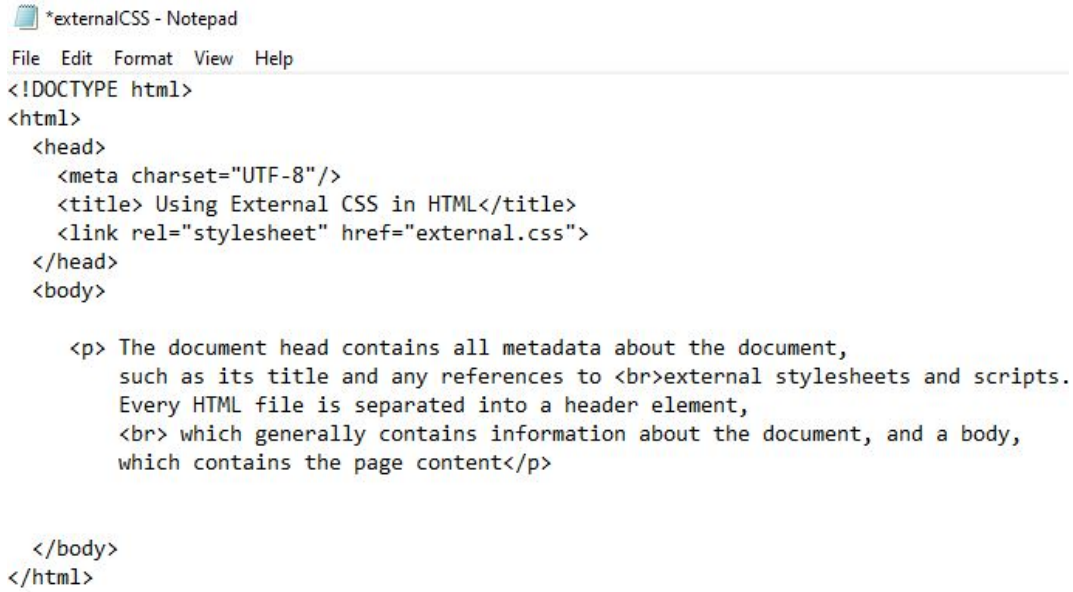
2. Reference an external stylesheet from the HTML

To store CSS outside of your HTML, save it in a plain-text file with a .css suffix, such as style.css. Then use a link element in the document head to reference the external CSS file, like so:

Type the following codes in text editor like Notepad and save it as" external.css" within the folder:



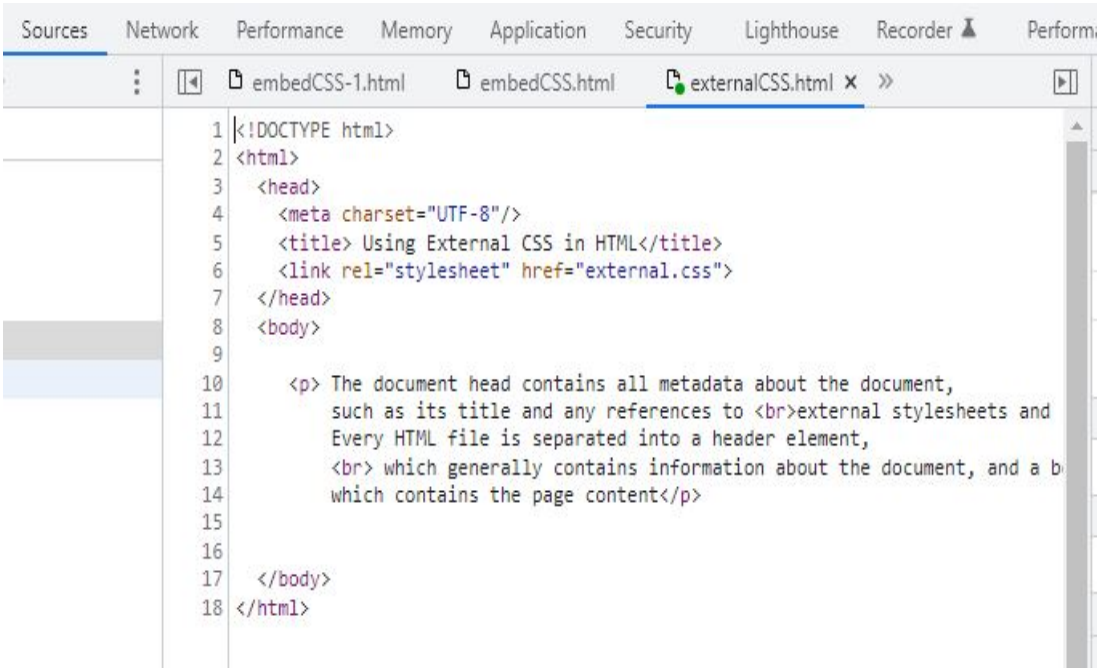
Type the following codes in text editor like Notepad:



After saving the html file and the opening the html file by using Chrome, you'll get:

The document head contains all metadata about the document, such as its title and any references to external stylesheets and scripts. Every HTML file is separated into a header element, which generally contains information about the document, and a body, which contains the page content

You can also view source codes by using Developer tools in Chrome:



Project 4. Answering questions and reading (Please do this at your home by using your own computer)

1. Read **perception in visualization** and give your comments**:**
   https://www.csc2.ncsu.edu/faculty/healey/PP/index.html

2. Give one or two examples in which you can recall or look at encoding data visualization by using perception principles, and write brief comments in which encoding approaches are particularly valuable.

Project 5. Creating a Professional Report

Summarize the above experiments procedure, results, answering questions and screenshots (project 1, 2, 3, 4) into one report. Your report is the assignment that is required to be submitted for evaluation on week 4. Create a report by following below steps.

Please note that

- as you may have a different version of Microsoft Word, the **following steps are just for your references.**

You can add a chapter called Chapter 2 in your previous report.

1. Open your last week's report and find the end of last week's report.
2. Copy this week's related experimental results, your findings and Screenshots, and paste them at the end of last week's report.
3. Delete original Table of Content you created.
4. Select all content, align all text to both left and right margin
5. Use shortcut key approach, generate Chapter 2: **Fundamentals of Web Technology: HTML, CSS**
6. Then use shortcut key approach to generate proper sub-chapters for this week's lab work.
7. Insert Table of Contents to your report.